Mekaneck: A Substrate-Neutral Language for Individuation-Structured Inquiry

Grammar, Type System, and Operational Semantics of `.mck`

Kundai Farai Sachikonye
Technical University of Munich
kundai.sachikonye@tum.de

August 16, 2026

Abstract

We define *Mekaneck* (source extension `.mck`), a small language whose sole computational primitive expresses the individuation of a target from a substrate by explicit exclusion, and whose termination condition is the exhaustion of reachable outcomes rather than the crossing of a confidence threshold. The language has one binding form, one primitive `seek` with a *mandatory* exclusion clause, and a fixed set of substrate declarations that bind an experimental domain to the primitive through four obligations: what the receivers are, what observable is recorded, what counts as a discriminating event, and how the irreducible floor is estimated. We give the complete lexical and context-free grammar, a type system with five types and two non-standard rules—a *positivity* rule rejecting programs that assert an attainable zero residual, and a *coherence* rule rejecting claims grounded on fewer than three mutually supporting independent sources—and a small-step operational semantics over a configuration comprising a substrate binding, a value store, and a monotone committed record. We prove progress and preservation for the type system, and hence that a well-typed program does not get stuck; we prove that evaluation is deterministic given the substrate and the sequence of catalyst outcomes, so that non-determinism enters only through substrate responses; we prove that the committed record is strictly monotone, so a program never returns to a previous configuration and its provenance order is intrinsic; we prove that every well-typed `seek` over a finite catalyst registry terminates, and that it terminates in exactly one of two typed outcomes—a *resolved* cell or an explicit *decline* carrying the plurality of incompatible cells found. We prove that the exclusion clause cannot be eliminated: a `seek` without it is not merely unidiomatic but untypable, because the target of an individuation is not determined by a positive description alone. We then show the language is substrate-neutral by exhibiting three bindings—an oscillatory-coherence substrate, a discrete-transition substrate, and a latency substrate—each requiring only the four obligations, and we identify precisely the condition on a substrate under which its discriminating events carry no information, so that a binding may be diagnosed as uninformative rather than the inquiry being reported as negative.

Keywords: domain-specific language; operational semantics; type soundness; individuation by exclusion; closure-based termination; substrate neutrality; scientific workflow languages.

Contents

1 Introduction

1.1 The design question

Languages for scientific computation generally supply a rich vocabulary of operations over a domain and leave the structure of inquiry implicit in how a user composes them. A script computes some quantities, applies a test, and—by a convention external to the language—stops when a number falls below a threshold. The structure of the inquiry lives in the analyst's head and in comments.

Mekaneck inverts this. It supplies almost no domain vocabulary and instead makes the structure of inquiry the only thing the language expresses. There is one computational primitive. It states what is being individuated, what it is being individuated *against*, by what means, and under what condition the individuation is complete. Everything domain-specific enters through a substrate binding that satisfies four obligations and is otherwise opaque to the language.

Two commitments distinguish the design, and both are enforced by the type system rather than by convention.

Exclusion is mandatory. A `seek` must state what its target is being told apart from. This is not stylistic. Theorem 4.5 shows that a positive description alone does not determine a target, so a `seek` lacking an exclusion clause has no well-defined denotation and is rejected at compile time.

Termination is by exhaustion, not by threshold. A `seek` completes when no remaining catalyst can carry it to an outcome outside the set already reached. Theorem 6.9 shows this condition is not obtainable by lowering a threshold, and Theorem 6.10 shows it resolves into exactly two typed outcomes, one of which is an explicit declination.

1.2 What the language is not

Mekaneck is not a general-purpose language and has no facilities for arithmetic beyond what the substrate supplies, no user-defined functions, no recursion, and no I/O other than through substrate bindings. It is a coordination language for inquiry, in the sense that a build language is a coordination language for compilation: the work happens elsewhere, and the language expresses only its structure. A program that needs to compute a spectral estimate calls a substrate that computes it; the language sees an opaque value.

1.3 Relation to existing work

The formal apparatus is standard: a context-free grammar in the style of Aho et al. [1], a type system presented as inference rules with progress and preservation in the manner of Wright and Felleisen [15] and Pierce [10], and a small-step structural operational semantics following Kahn [6], Plotkin [11]. The novelty is not in the metatheory but in what the rules enforce.

Scientific workflow languages—Make and its lineage, and the modern dataflow workflow engines [2, 5, 7]—express dependency and let the user supply the stopping condition. Statistical workflow systems embed stopping rules in libraries rather than in the language. Sequential-analysis stopping rules [3, 8, 14] halt when an expected gain or a likelihood ratio crosses a boundary; Theorem 6.9 distinguishes our criterion from all such rules. The requirement of three mutually supporting independent sources (Section 5) is related to the triangulation heuristics discussed in methodological work on convergent evidence [9] and to the treatment of conflicting evidence in belief functions [12], but is here a typing rule with a proof rather than a recommendation. The insistence that the language report a plurality when evidence conflicts, rather than select a representative, parallels the "honest decline" available in conformal and abstention-based prediction [4, 13].

2 The substrate interface

Everything domain-specific enters through a substrate. A substrate is required to discharge exactly four obligations, and the language depends on nothing else about it.

Definition 2.1 (Substrate)**.** A *substrate* $\mathcal{D}$ is a four-tuple of obligations:

(S1) **Receivers.** A finite set $R(\mathcal{D})$ of bounded systems, each of which is the unit to which a floor attaches.

(S2) **Observable.** For each receiver, a finite sequence of *states*, each carrying a real-valued uncertainty S and a label drawn from a finite label set.

(S3) **Events.** A rule determining which ordered pairs of states constitute *discriminating events*, and a type for each such event drawn from a finite set.

(S4) **Floor.** An estimator returning $\beta(\mathcal{D}, r) > 0$ for each receiver r, the irreducible residual below which uncertainty does not fall.

Remark 2.2 (The floor obligation is the substantive one)*.* (S1)–(S3) are bookkeeping: they say what the units are, what is measured, and what counts as a step. (S4) is a commitment with empirical content. A substrate that discharges it by returning the minimum observed S has discharged it trivially—the value is positive whenever the sample is, and carries no information. A substrate that discharges it by estimating the asymptote of separation cost as the sample grows has made a claim capable of being wrong. The language cannot tell these apart, and we therefore require a substrate to declare which estimator it uses, so that a reader of a `.mck` program can see it. Section 8 gives the diagnostic that detects the trivial case from data.

Definition 2.3 (Catalyst)**.** A *catalyst* is a named means of advancing an inquiry: a computation over substrate data, a query to an external source, or an inferential procedure. Catalysts are

opaque to the language, which knows only their names, their declared independence relation (Section 5), and the outcomes they return.

Definition 2.4 (Cell)**.** A *cell* is an equivalence class of outcomes under a substrate-supplied indistinguishability relation. Cells, not points, are the values a `seek` returns.

Remark 2.5 (Why the return type is a cell). The language cannot return a point-valued answer, because the floor obligation (S4) asserts a positive irreducible residual and a point-valued answer would assert a zero residual. This is enforced by the positivity rule of Section 4 and is the reason the type `Point` does not exist in the language.

3 Lexical structure and grammar

3.1 Token classes

Definition 3.1 (Tokens)**.** The lexer recognises: *keywords* `substrate`, `receivers`, `observable`, `events`, `floor`, `catalyst`, `independent`, `let`, `seek`, `excluding`, `via`, `until`, `closure`, `report`, `resolved`, `decline`, `record`; *identifiers* matching `[A-Za-z_][A-Za-z0-9_]*` that are not keywords; *string literals* delimited by double quotes; *numeric literals* in decimal notation; *punctuation* `{ } ( ) , ; : =`; and *comments* from `#` to end-of-line, discarded. Whitespace is insignificant except as a token separator.

3.2 Context-free grammar

⟨*program*⟩ ::= ⟨*decl*⟩*

⟨*decl*⟩ ::= ⟨*substrate-decl*⟩ | ⟨*catalyst-decl*⟩ | ⟨*let-decl*⟩ | ⟨*report-decl*⟩

⟨*substrate-decl*⟩ ::= '`substrate`' ⟨*ident*⟩ '`{`' '`receivers`' '`:`' ⟨*expr*⟩ '`;`' '`observable`' '`:`' ⟨*expr*⟩ '`;`' '`events`' '`:`' ⟨*expr*⟩ '`;`' '`floor`' '`:`' ⟨*expr*⟩ '`;`' '`}`'

⟨*catalyst-decl*⟩ ::= '`catalyst`' ⟨*ident*⟩ '`:`' ⟨*expr*⟩ ['`independent`' ⟨*ident-list*⟩] '`;`'

⟨*let-decl*⟩ ::= '`let`' ⟨*ident*⟩ '`=`' ⟨*seek-expr*⟩ '`;`'

⟨*report-decl*⟩ ::= '`report`' ⟨*ident*⟩ '`;`'

⟨*seek-expr*⟩ ::= '`seek`' ⟨*expr*⟩ '`excluding`' ⟨*expr*⟩ ['`via`' '`(`' ⟨*ident-list*⟩ '`)`'] '`until`' ⟨*term-cond*⟩

⟨*term-cond*⟩ ::= '`closure`'

⟨*ident-list*⟩ ::= ⟨*ident*⟩ ('`,`' ⟨*ident*⟩)*

⟨*expr*⟩ ::= ⟨*ident*⟩ | ⟨*string*⟩ | ⟨*number*⟩ | ⟨*ident*⟩ '`(`' ⟨*arg-list*⟩? '`)`'

⟨*arg-list*⟩ ::= ⟨*expr*⟩ ('`,`' ⟨*expr*⟩)*

Proposition 3.2 (The grammar is unambiguous)**.** *The grammar of Section 3 is LL(1) and therefore unambiguous.*

Proof. Each production for a non-terminal begins with a distinct terminal: the four `<decl>` alternatives begin with the distinct keywords `substrate`, `catalyst`, `let`, `report`; the `<seek-expr>` clauses are introduced by the distinct keywords `seek`, `excluding`, `via`, `until`; the optional clauses `via` and `independent` are distinguished from what follows by their leading keyword; and `<expr>` alternatives are distinguished by token class, with the function-call form disambiguated by a one-token lookahead for `(`. Hence a single token of lookahead selects a unique production at every point, so the grammar is LL(1), and LL(1) grammars are unambiguous [1]. □

Remark 3.3 (The exclusion clause is not optional in the grammar). Note that `via` and `independent` are bracketed as optional in the grammar while `excluding` is not. This is deliberate and is the syntactic reflection of Theorem 4.5: a `seek` without an exclusion clause is not a parse error to be recovered from but a form the language does not contain.

3.3 An example program

```
# Individuate a coherence regime against the rest of the record.
substrate Osc {
  receivers : recordings("cohort-A");
  observable : coherence_index();
  events : label_change();
  floor : asymptotic_separation(); # not sample_minimum()
}

catalyst spectral : band_decomposition() independent surrogate, phase;
catalyst surrogate : phase_randomised() independent spectral, phase;
catalyst phase : locking_value() independent spectral, surrogate;

let regime =
  seek target_state("high-coherence")
  excluding all_other_states()
  via (spectral, surrogate, phase)
  until closure;

report regime;
```

The program declares a substrate discharging the four obligations, three mutually independent catalysts, and one `seek` whose target is individuated against the explicitly named remainder. It terminates by closure, and `report` emits either a resolved cell or a declination.

4 The type system

4.1 Types

Definition 4.1 (Types)**.** The types are

$$T \;::=\; \texttt{Substrate} \mid \texttt{Catalyst} \mid \texttt{Cell} \mid \texttt{Outcome} \mid \texttt{Floor}.$$

There is no type of points: the language cannot express a residue-free answer.

Definition 4.2 (Outcome)**.** $\texttt{Outcome} = \texttt{Resolved}(\texttt{Cell}) \mid \texttt{Declined}(\{\texttt{Cell}\})$, a sum whose second summand carries a set of at least two mutually incompatible cells.

4.2 Judgements and standard rules

A typing environment Γ maps identifiers to types. The judgement $\Gamma \vdash e : T$ is derived by the following rules, in which T-Var, T-Sub, T-Cat are standard.

$$\text{T-Var}\quad \frac{x : T \in \Gamma}{\Gamma \vdash x : T} \qquad \text{T-Sub}\quad \frac{\Gamma \vdash r, o, v, f \text{ well-formed}}{\Gamma \vdash \texttt{substrate}\,\{r; o; v; f\} : \texttt{Substrate}}$$

$$\text{T-Cat}\quad \frac{\Gamma \vdash e : \_ \qquad \iota \subseteq \mathrm{dom}(\Gamma)}{\Gamma \vdash \texttt{catalyst}\ c : e\ \texttt{independent}\ \iota : \texttt{Catalyst}}$$

4.3 The two non-standard rules

Definition 4.3 (Positivity rule)**.**

$$\textsc{T-Seek-Pos}\quad \frac{\Gamma \vdash t : \texttt{Cell} \quad \Gamma \vdash x : \texttt{Cell} \quad \Gamma \vdash \beta : \texttt{Floor} \quad \beta > 0}{\Gamma \vdash \textbf{seek}\ t\ \textbf{excluding}\ x\ \cdots : \texttt{Outcome}}$$

A `seek` is well typed only if the substrate's floor is positive. A program asserting an attainable zero residual—by declaring a floor of zero, or by requesting a point-valued target—does not type.

Definition 4.4 (Coherence rule)**.**

$$\textsc{T-Seek-Coh}\quad \frac{\Gamma \vdash c_i : \texttt{Catalyst}\ (1 \le i \le k) \quad k \ge 3 \quad \mathrm{MutIndep}(c_1, \ldots, c_k)}{\Gamma \vdash \textbf{seek}\ \ldots\ \textbf{via}\ (c_1, \ldots, c_k)\ \textbf{until}\ \texttt{closure} : \texttt{Outcome}}$$

where MutIndep holds when the declared independence relation contains every unordered pair. A `seek` with an explicit `via` clause is well typed only if it names at least three mutually independent catalysts.

Section 5 justifies the threshold of three.

4.4 Necessity of the exclusion clause

Theorem 4.5 (Exclusion cannot be eliminated)**.** *Let a target be specified by a positive description alone—a predicate asserting properties the target has—within a substrate whose state space contains no distinguished element. Then the description does not determine a unique cell, unless the description already names the complement it is to be distinguished from. Consequently a* `seek` *without an exclusion clause has no unique denotation, and the language is right to exclude the form.*

Proof. A cell is an equivalence class of outcomes (Theorem 2.4); to determine a cell is to determine, of each outcome, whether it belongs. A positive description asserts properties of members. Suppose two distinct cells $A \neq A'$ both satisfy every asserted property—which is possible whenever the description does not mention a property on which A and A' differ. Then the description determines neither, and no appeal to a distinguished element resolves the tie, since by hypothesis none exists. To force uniqueness the description must assert a property that A has and A' lacks; asserting such a property is asserting what the target is *not*—namely, not the thing lacking it—which is an exclusion. Hence either the specification is incomplete or it contains an exclusion, and a form with no exclusion clause cannot in general denote uniquely. □

Corollary 4.6 (Compile-time rejection)**.** *A parser for the grammar of Section 3 rejects* **seek** *t* **via** ... *without* `excluding`*, and no typing derivation exists for such a term.*

5 Why three catalysts

Definition 5.1 (Support structure)**.** Given catalysts $c_1, \ldots, c_k$ bearing on a target, the *support structure* is the directed graph on $\{c_1, \ldots, c_k\}$ with an edge $c_i \to c_j$ when the outcome of c_i is used to license the outcome of c_j. A conclusion is *robust* if it survives the removal of any single catalyst.

Theorem 5.2 (Linear support is not robust)**.** *If the support structure is acyclic, the conclusion is not robust: there is a catalyst whose removal leaves some other catalyst's contribution unlicensed, and by induction the chain collapses.*

Proof. An acyclic finite digraph has a vertex of in-degree zero, say c_s: its contribution is licensed by nothing internal to the structure. Removing the terminal vertex of a maximal path leaves its predecessor's contribution unused, and removing c_s leaves every vertex reachable only through it unlicensed. In either case some contribution's licensing is lost, so the conclusion does not survive the removal of that single catalyst, and is not robust by Theorem 5.1. □

Theorem 5.3 (Three is the minimum)**.** *A robust support structure contains a directed cycle of length at least three. Cycles of length one (self-support) and two (mutual support of a pair) do not confer robustness. Conversely, three catalysts each supported by the other two form a robust structure.*

Proof. By Theorem 5.2 a robust structure is not acyclic, so it contains a cycle. A 1-cycle is a catalyst citing itself, which licenses nothing not already assumed. For a 2-cycle $\{c_1, c_2\}$: removing either leaves the other supported by nothing, so the conclusion does not survive a single removal and is not robust. Hence the shortest cycle compatible with robustness has length ≥ 3. Conversely, in a 3-cycle with all mutual edges, removing any one catalyst leaves the remaining two still mutually supporting via the edge between them, so a licensed contribution survives and the structure is robust. □

Remark 5.4 (What the rule does and does not guarantee). Theorem 4.4 enforces a structural necessary condition, not sufficiency for a correct conclusion. Three catalysts that are declared independent but are not—sharing a common upstream data source, a common preprocessing step, or a common modelling assumption—satisfy the rule while violating its intent. The language cannot verify independence; it can only require that it be declared, so that the declaration is visible in the source and auditable. We regard this as the honest division of labour: mechanical enforcement of the structure, human responsibility for the premise.

6 Operational semantics

6.1 Configurations

Definition 6.1 (Configuration)**.** A *configuration* is a triple $\langle e,\ \Sigma,\ \rho\rangle$ where e is the term being evaluated, Σ is the value store mapping identifiers to values, and $\rho \in \mathbb{N}$ is the *committed record*, a count of catalyst invocations performed.

Definition 6.2 (Reachable-cell set)**.** For a `seek` in progress, $\mathcal{C}$ denotes the set of cells reached by catalysts invoked so far.

6.2 Reduction rules

We write $\Sigma(c) \Downarrow z$ for "invoking catalyst c against the substrate yields cell z". The rules are:

$$\text{E-Invoke}\quad \frac{c \in \Gamma_{\text{avail}} \qquad \Sigma(c) \Downarrow z}{\langle \textbf{seek}\ t\ \textbf{excluding}\ x\ \textbf{via}\ \Gamma_{\text{avail}}\ \textbf{until}\ \texttt{closure},\ \Sigma,\ \rho\rangle \longrightarrow \langle \textbf{seek}\ t\ \textbf{excluding}\ x\ \textbf{via}\ \Gamma_{\text{avail}} \setminus \{c\}\ \textbf{u}}$$

$$\text{E-Close-Res}\quad \frac{\forall c \in \Gamma_{\text{avail}} :\ \Sigma(c) \Downarrow z \Rightarrow z \in \mathcal{C} \qquad |\mathcal{C}| = 1}{\langle \textbf{seek}\ \ldots,\ \Sigma,\ \rho\rangle \longrightarrow \langle \texttt{Resolved}(z_0),\ \Sigma,\ \rho\rangle}$$

$$\text{E-Close-Dec}\quad \frac{\forall c \in \Gamma_{\text{avail}} :\ \Sigma(c) \Downarrow z \Rightarrow z \in \mathcal{C} \qquad |\mathcal{C}| \geq 2}{\langle \textbf{seek}\ \ldots,\ \Sigma,\ \rho\rangle \longrightarrow \langle \texttt{Declined}(\mathcal{C}),\ \Sigma,\ \rho\rangle}$$

E-Invoke consumes one available catalyst, adds its cell to the reached set, and increments the record. The two closure rules fire when no remaining catalyst can add a cell, and are distinguished solely by the cardinality of $\mathcal{C}$.

6.3 Metatheory

Theorem 6.3 (Progress)**.** *If $\Gamma \vdash e$: `Outcome` and e is not of the form `Resolved`$(\cdot)$ or `Declined`$(\cdot)$, then for any store Σ and record ρ there is a configuration to which $\langle e, \Sigma, \rho\rangle$ steps.*

Proof. A well-typed non-value term of type `Outcome` is a `seek` by Theorems 4.3 and 4.4, the only rules concluding that type. Consider its available set Γ_{avail}. If some available catalyst yields a cell outside $\mathcal{C}$, then E-Invoke applies with that catalyst. Otherwise every available catalyst yields a cell in $\mathcal{C}$; then $\mathcal{C}$ is nonempty—by T-Seek-Coh at least three catalysts were named and, if none has yet been invoked, E-Invoke applies to any of them—so exactly one of E-Close-Res ($|\mathcal{C}| = 1$) or E-Close-Dec ($|\mathcal{C}| \geq 2$) applies. In every case a rule fires, so the term is not stuck. □

Theorem 6.4 (Preservation)**.** *If $\Gamma \vdash e : T$ and $\langle e, \Sigma, \rho\rangle \longrightarrow \langle e', \Sigma', \rho'\rangle$ then $\Gamma \vdash e' : T$.*

Proof. By cases on the rule applied. E-Invoke yields a `seek` term differing from its predecessor only in the available set and the store; the premises of T-Seek-Pos concern the floor, unchanged, and those of T-Seek-Coh concern the catalysts named in the `via` clause at the time of typing, which are not re-checked at each step—the rule constrains the program text, not the residual available set. Hence the reduct types at `Outcome`. E-Close-Res yields `Resolved`(z_0) with z_0 : `Cell`, of type `Outcome` by Theorem 4.2; E-Close-Dec yields `Declined`$(\mathcal{C})$ with $|\mathcal{C}| \geq 2$, likewise of type `Outcome`. In all cases the type is preserved. □

Corollary 6.5 (Soundness)**.** *A well-typed program does not get stuck: it either reduces indefinitely or reaches a value of type `Outcome`. Combined with Theorem 6.8 below, every well-typed program reaches such a value in finitely many steps.*

Proof. Immediate from Theorems 6.3 and 6.4 by the standard syntactic argument [15]. □

Theorem 6.6 (Monotone record and non-return)**.** *The record ρ is non-decreasing along any reduction and strictly increases at each E-Invoke. Two configurations with identical terms and stores but different records are distinct; hence no program returns to a previous configuration.*

Proof. E-Invoke increments ρ; the closure rules leave it unchanged and produce values, from which no further step is taken. No rule decrements ρ. Configurations with records $r < r'$ are separated by the predicate $\rho \leq r$, hence distinct. A return would require ρ to decrease, which no rule permits. □

Theorem 6.7 (Determinism modulo substrate)**.** *Fix a substrate and fix, for each catalyst, the cell it yields. Then reduction is deterministic up to the order of E-Invoke steps, and the final `Outcome` is independent of that order.*

Proof. The only choice in the semantics is which available catalyst E-Invoke consumes. Each invocation adjoins $\Sigma(c) \Downarrow z$ to $\mathcal{C}$, and set union is associative, commutative, and idempotent, so the reached set after all catalysts have been consumed is $\{\Sigma(c) \Downarrow z : c \in \Gamma_{\text{avail}}\}$ regardless of order. The closure rules are selected by $|\mathcal{C}|$, a function of that set. Hence the final outcome is order-independent, and any residual non-determinism arises solely from the substrate's responses. □

Theorem 6.8 (Termination)**.** *Every well-typed `seek` over a finite catalyst registry reaches an `Outcome` in at most $|\Gamma_{\text{avail}}|$ invocations.*

Proof. Each E-Invoke removes one catalyst from the available set, which is finite, so at most $|\Gamma_{\text{avail}}|$ such steps occur. When the set is empty the premise of the closure rules—that every available catalyst yields a cell already in $\mathcal{C}$—is vacuously satisfied, so a closure rule fires and a value is produced. Hence termination in at most $|\Gamma_{\text{avail}}|$ invocations, and possibly earlier if closure is detected before exhaustion. □

Theorem 6.9 (Closure is strictly stronger than a threshold)**.** *For any threshold θ on attained uncertainty there is a substrate and program in which a partial invocation attains uncertainty below θ while an uninvoked catalyst would reach a cell outside $\mathcal{C}$; the threshold criterion terminates, the closure criterion does not.*

Proof. Take a substrate with two catalysts c_A, c_B reaching incompatible cells $z_A \neq z_B$, with the uncertainty attained at z_A below θ. Invoking c_A alone satisfies the threshold. But c_B is available and $z_B \notin \mathcal{C} = \{z_A\}$, so the premise of both closure rules fails and the closure criterion requires E-Invoke on c_B. Hence the two criteria differ on this program, with closure requiring strictly more. □

Theorem 6.10 (Dichotomy of outcomes)**.** *Every terminating* `seek` *produces either* `Resolved`(z) *or* `Declined`$(\mathcal{C})$ *with* $|\mathcal{C}| \geq 2$*, and never both.*

Proof. By Theorem 6.8 a closure rule fires. The two rules have premises $|\mathcal{C}| = 1$ and $|\mathcal{C}| \geq 2$, which are mutually exclusive and, since $\mathcal{C}$ is nonempty at closure (at least one catalyst has been invoked by Theorem 6.3), jointly exhaustive. □

Corollary 6.11 (Declination is a first-class result)**.** *A program whose evidence is genuinely conflicting terminates normally, returning the plurality of incompatible cells. This is a value of type* `Outcome`*, not an error, and* `report` *emits it as such.*

7 Substrate neutrality

The language depends on a substrate only through the four obligations of Theorem 2.1. We exhibit three bindings of different character to show the obligations are dischargeable without altering the language.

Example 7.1 (Oscillatory-coherence substrate)**.** Receivers are recordings. The observable is a coherence index in $[0, 1]$ computed from multi-channel phase relationships, mapped to uncertainty by an order-reversing affine map so that higher coherence is lower uncertainty. Events are transitions between adjacent segments carrying different labels, typed by the ordered label pair. The floor is estimated by the asymptotic method of Section 8.

Example 7.2 (Discrete-transition substrate)**.** Receivers are experimental conditions. States are members of a small finite class set with an associated occupancy and dwell statistic; uncertainty is derived from occupancy. Events are class-to-class transitions, typed by the ordered class pair. This binding is appropriate when the underlying record is a transition matrix rather than a time series.

Example 7.3 (Latency substrate)**.** Receivers are participants. States are trials labelled by condition, with uncertainty an affine function of response latency. Events are the addition of an information source to a condition, typed by the source added. This binding yields a zero-free-parameter prediction: the power of a compound condition should equal the composition of the powers of its constituents.

Proposition 7.4 (Neutrality)**.** *No rule of Sections 3, 4 and 6 refers to any feature of a substrate beyond the four obligations. Consequently a program is well-typed and reduces identically under any substrate discharging them.*

Proof. Inspection of the rules: the grammar mentions substrate fields by keyword only; T-Sub requires well-formedness of the four fields and nothing else; T-Seek-Pos requires only $\beta > 0$, supplied by (S4); T-Seek-Coh concerns catalysts, not substrates; and the reduction rules use the substrate only through $\Sigma(c) \Downarrow z$, which is (S2) and (S3) mediated by a catalyst. No rule inspects the internal structure of a receiver, observable, event, or floor. □

8 Diagnostics: when a binding is uninformative

A binding may satisfy every obligation and still be incapable of supporting an inference. The language cannot detect this, but a diagnostic computed from substrate data can, and we specify it so that a negative result is attributed correctly.

Definition 8.1 (Type separation)**.** For a corpus of events with types, let Var_b be the variance across types of the per-type mean power and Var_w the mean within-type variance. The *separation* is $\eta = \mathrm{Var}_\mathrm{b}/(\mathrm{Var}_\mathrm{b} + \mathrm{Var}_\mathrm{w}) \in [0, 1]$.

Proposition 8.2 (Uninformative bindings)**.** *If $\eta \approx 0$, the per-type means are indistinguishable, predictions formed from them are constant at fixed cascade length, and no comparison among composition rules has discriminating power. A negative result obtained in this regime is evidence about the observable, not about the process.*

Proof. If all type means coincide at $\bar{\kappa}$, any prediction formed as a function of the type means alone depends only on the number of events composed; at fixed length it is constant, so its sample variance vanishes and any correlation with measured outcomes is undefined. Competing rules computed from the same means are likewise constant and cannot be separated. □

Remark 8.3 (The floor diagnostic). The second diagnostic concerns obligation (S4). Two estimators are distinguishable from data: the sample-minimum estimator returns the least observed uncertainty and is positive whenever the sample is nonempty; the asymptotic estimator fits the limiting value of separation cost as the sample grows and can return a value statistically indistinguishable from zero. Only the second can fail. A substrate binding should declare which it uses, and a reader should treat a positivity claim from the former as carrying no evidential weight. Section 9 tests both estimators against generating processes with and without a genuine floor.

9 Numerical verification

9.1 Method

We implement a lexer, parser, type checker, and evaluator for the language as specified, and exercise them against constructed programs and synthetic substrates. The following are checked.

Grammar and parsing. Well-formed programs parse; programs omitting the `excluding` clause are rejected at parse time (Theorem 4.6).

Typing. Programs declaring a non-positive floor are rejected by T-Seek-Pos; programs naming fewer than three catalysts, or three that are not pairwise declared independent, are rejected by T-Seek-Coh; conforming programs type.

Progress and preservation. Over randomly generated well-typed programs and substrates, every intermediate configuration is checked to be either a value or reducible, and the type of each reduct is checked to be `Outcome` (Theorems 6.3 and 6.4).

Determinism and termination. Each program is evaluated under randomised catalyst orderings; the final outcome is checked to be independent of order (Theorem 6.7), and the invocation count is checked against the bound of Theorem 6.8.

Record monotonicity. The record is checked to increase strictly at each invocation and never to decrease (Theorem 6.6).

Dichotomy. Substrates are constructed to force each branch: one in which all catalysts agree, yielding `Resolved`; one in which two catalysts reach incompatible cells, yielding `Declined`. We check that the latter terminates normally rather than raising (Theorem 6.11).

Closure versus threshold. The construction of Theorem 6.9 is instantiated, and we verify that a threshold rule stops after one catalyst while closure continues and reports a declination.

Diagnostics. Both floor estimators are applied to generating processes with and without a floor, and the separation η is computed for substrates of varying type separation, verifying that the uninformative regime is detected (Theorem 8.2).

9.2 What would count as a failure

A parse acceptance of an exclusion-free `seek` would falsify Theorem 4.6; an order-dependent outcome would falsify Theorem 6.7; a stuck configuration would falsify Theorem 6.3; a run exceeding the invocation bound would falsify Theorem 6.8; a raised exception on conflicting evidence would falsify Theorem 6.11. Results are recorded in machine-readable form alongside this paper.

9.3 Results

All five checks passed.

Parsing and typing. The reference program of Section 3 parsed and type-checked. The same program with the `excluding` clause deleted was rejected at parse time, as Theorem 4.6 requires: the rejection occurs before typing, because the form is absent from the grammar rather than ill-typed within it. Declaring a floor of 0 or of -3 was rejected by T-Seek-Pos; a `via` clause naming two catalysts was rejected by T-Seek-Coh; and a `via` clause naming three catalysts of which one was not declared mutually independent of the others was also rejected, showing the rule tests the independence relation and not merely the count.

Semantics. Over 500 randomly generated substrates, each evaluated under all $3! = 6$ orderings of the catalyst set—3000 evaluations in total—there were zero violations of progress or preservation, zero of determinism, zero of the termination bound, zero of record monotonicity, and zero of the outcome dichotomy. Every evaluation terminated in at most $|\Gamma_{\text{avail}}| = 3$ invocations, as Theorem 6.8 requires, and every configuration reduced to a value of type `Outcome`.

Both branches occur. Of the 500 substrates, 296 terminated in `Resolved` and 204 in `Declined`. We record this because a dichotomy theorem is compatible with one branch being unreachable in practice, and a language whose declination branch never fires would be a language with a threshold criterion in disguise. The near-balance here is an artefact of the substrate sampling—cell counts were drawn so that agreement and disagreement were both common—but it establishes that both branches are live and that declination arises from ordinary substrate configurations rather than from contrived ones.

Closure against a threshold. On the substrate in which two catalysts reach a common cell and the third reaches an incompatible one, with the common cell carrying an attained uncertainty of 5.0 against a threshold of $\theta = 10.0$, the threshold procedure halted after 1 invocation reporting `Resolved(cellA)`, while the closure procedure performed 2 invocations and reported `Declined({cellA, cellB})`. The two procedures returned incompatible verdicts on identical evidence, which is the concrete content of Theorem 6.9.

Remark 9.1 (The threshold procedure stopped before the disagreement was reachable). The detail worth drawing out is *when* the threshold procedure halted: after one invocation, not two. It stopped as soon as a single source reported an outcome it found confident, and therefore never invoked the source that would have contradicted it—not because that source was unavailable, but because the criterion had already been satisfied. This is the failure mode of **??** realised in a concrete evaluation: internal consistency of one line of evidence is sufficient to satisfy a threshold, and a threshold cannot express the requirement that the remaining sources be consulted. Closure expresses exactly that requirement, and does so without reference to any magnitude—the closure rules of Section 6 inspect only whether the reached set would grow, never how confident any outcome is.

Diagnostics. The separation statistic distinguished an informative from an uninformative binding: $\eta = 0.992$ for a substrate whose four event types had well-spaced mean powers, against $\eta = 2.20 \times 10^{-3}$ for a substrate whose type means agreed to three significant figures, the latter falling below the flagging threshold of Theorem 8.2.

The floor estimators were compared in two sampling regimes, Table 1. In both, the sample minimum returned a strictly positive value for a process with no floor—2.82×10^{-11} at realistic

sample sizes and 6.08×10^{-17} at large ones—and was biased upward on the floored process. The asymptotic estimator recovered the true floor to within 0.002 and returned a negative value on the unfloored process in both regimes. The contrast is the practical justification for requiring a substrate to declare which estimator discharges obligation (S4): the two agree on a process that has a floor and disagree in kind on one that does not, and only the second can report the disagreement.

Table 1: Floor estimators on generating processes with true floor 10 and with no floor.

Regime	Estimator	Floored ($\beta = 10$)	Unfloored ($\beta = 0$)
$n = 50$–400	Sample minimum	10.0000000010	2.82×10^{-11}
	Asymptotic	9.9981	-1.76×10^{-5}
$n = 50$–4000	Sample minimum	10.0000000000	6.08×10^{-17}
	Asymptotic	10.0000	-2.32×10^{-3}

10 Scope and limitations

The language expresses the structure of an inquiry and delegates all computation to substrates; it is therefore useless without a substrate and makes no claim to compute anything itself. The coherence rule enforces a structural condition and cannot verify the independence it requires (Theorem 5.4). The closure criterion is only as good as the catalyst registry: closure over an impoverished registry is easily attained and means only that the registry is exhausted, not that the question is settled—the criterion quantifies over available catalysts, not over conceivable ones. The positivity rule prevents a program from asserting a zero residual but cannot prevent a substrate from discharging the floor obligation trivially; the diagnostic of Theorem 8.3 is offered because the type system cannot do this work. Finally, the semantics treats catalyst invocation as returning a determinate cell; substrates whose responses are noisy require the indistinguishability relation of Theorem 2.4 to absorb that noise, and the choice of that relation is a substantive modelling decision the language does not make.

10.1 Conclusion

Mekaneck has one primitive, two non-standard typing rules, and three reduction rules. The primitive requires an inquiry to say what its target is being told apart from, and the type system rejects programs that do not; the termination condition requires an inquiry to exhaust its available means rather than to cross a threshold, and the semantics guarantees it does so in finitely many steps; and the outcome type admits an explicit declination, so that conflicting evidence terminates a program normally instead of forcing a choice the evidence does not support. The metatheory is standard—progress, preservation, determinism modulo substrate, termination—and the substrate interface is narrow enough that the same program runs unchanged over substrates of quite different character. What the language cannot do, it declines to pretend to do: it cannot verify independence, cannot validate a floor estimator, and cannot know whether the catalysts available to it are the ones that matter.

References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, Boston, 2nd edition, 2006.

[2] Peter Amstutz, Michael R. Crusoe, Nebojša Tijanić, Brad Chapman, John Chilton, Michael Heuer, Andrey Kartashov, Dan Leehr, Hervé Ménager, Maya Nedeljkovich, Matt Scales, Stian

Soiland-Reyes, and Luka Stojanovic. Common workflow language, v1.0. *Figshare*, 2016. doi: 10.6084/m9.figshare.3115156.v2.

[3] Kathryn Chaloner and Isabella Verdinelli. Bayesian experimental design: A review. *Statistical Science*, 10(3):273–304, 1995.

[4] C. K. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1):41–46, 1970.

[5] Paolo Di Tommaso, Maria Chatzou, Evan W. Floden, Pablo Prieto Barja, Emilio Palumbo, and Cedric Notredame. Nextflow enables reproducible computational workflows. *Nature Biotechnology*, 35(4):316–319, 2017.

[6] Gilles Kahn. Natural semantics. In *STACS 87: 4th Annual Symposium on Theoretical Aspects of Computer Science*, volume 247 of *Lecture Notes in Computer Science*, pages 22–39. Springer, 1987.

[7] Johannes Köster and Sven Rahmann. Snakemake—a scalable bioinformatics workflow engine. *Bioinformatics*, 28(19):2520–2522, 2012.

[8] D. V. Lindley. On a measure of the information provided by an experiment. *The Annals of Mathematical Statistics*, 27(4):986–1005, 1956.

[9] Marcus R. Munafò and George Davey Smith. Robust research needs many lines of evidence. *Nature*, 553(7689):399–401, 2018.

[10] Benjamin C. Pierce. *Types and Programming Languages*. MIT Press, Cambridge, MA, 2002.

[11] Gordon D. Plotkin. A structural approach to operational semantics. Technical Report DAIMI FN-19, Computer Science Department, Aarhus University, 1981.

[12] Glenn Shafer. *A Mathematical Theory of Evidence*. Princeton University Press, Princeton, NJ, 1976.

[13] Vladimir Vovk, Alexander Gammerman, and Glenn Shafer. *Algorithmic Learning in a Random World*. Springer, New York, 2005.

[14] Abraham Wald. Sequential tests of statistical hypotheses. *The Annals of Mathematical Statistics*, 16(2):117–186, 1945.

[15] Andrew K. Wright and Matthias Felleisen. A syntactic approach to type soundness. *Information and Computation*, 115(1):38–94, 1994.